

VIETNAM NATIONAL UNIVERSITY — HO CHI MINH CITY
UNIVERSITY OF SCIENCE
Faculty of Information Technology



PRACTICAL REPORT 1

COLOR COMPRESSION VIA K-MEANS CLUSTERING

An unsupervised learning approach to RGB palette reduction

Course: **MTH00051 — Applied Mathematics and Statistics**

STUDENT

Nguyễn Thế Hiển

Student ID: 22127107 · Class: 22CLC08

INSTRUCTORS

Mr. Vũ Quốc Hoàng

Mr. Nguyễn Văn Quang Huy

Mr. Nguyễn Ngọc Toàn

Mrs. Phan Thị Phương Uyên

Ho Chi Minh City, 2024

Acknowledgments

This practical exercise forms part of MTH00051 — Applied Mathematics and Statistics, a foundational course for students of Information Technology. Through the Color Compression project, I had the opportunity to connect clustering theory with a concrete computational application.

I gratefully acknowledge the guidance of Mr. Vũ Quốc Hoàng, Mr. Nguyễn Văn Quang Huy, Mr. Nguyễn Ngọc Toàn, and Mrs. Phan Thị Phương Uyên. Their lectures and laboratory support clarified both the mathematical principles and the engineering constraints of the assignment. I also thank my classmates in 22CLC08 for constructive discussion throughout the course.

Any remaining errors in the implementation or in this report are my own.

Abstract

Digital RGB images admit up to 256^3 distinct colors, which makes naive storage expensive. Color compression (color quantization) seeks a compact palette of k representative colors while preserving the visual identity of the scene. In this report, each pixel is modeled as a point in $\mathbb{R}^3$ and clustered by the K-Means algorithm implemented from first principles in Python.

We formalize the optimization objective as minimization of the within-cluster sum of squares, describe Lloyd's alternating minimization procedure, and compare two centroid-initialization schemes required by the course specification: *random* and *in_pixels*. Qualitative experiments for $k \in \{3, 5, 7\}$ show that larger palettes recover finer tonal structure, whereas initialization mainly affects which local minimum is attained. The study illustrates how a classical unsupervised method links Euclidean geometry, iterative optimization, and practical image processing.

Keywords: color quantization; K-Means clustering; Lloyd iteration; Euclidean distance; image compression; unsupervised learning.

Contents

1. Introduction	4
2. Problem Formulation	4
3. Theoretical Background	5
3.1 K-Means as an Optimization Procedure	5
3.2 Centroid Initialization Strategies	5
4. Implementation Idea and Description	6
4.1 Software Environment	6
4.2 Implementation Idea (K-Means Recipe)	6
4.3 Detailed Function Descriptions	7
4.4 Testing Procedure	9
4.5 Main Program Workflow	9
5. Experimental Results and Commentary	10
5.1 Experimental Setup	10
5.2 Visual Comparison for $k = 3, 5, 7$	10
5.3 Qualitative Summary	12
6. Discussion	12
7. Conclusion	13
References	13

List of Figures

Figure 1. Original test image prior to compression.

Figure 2. Reconstructions for $k = 3$ (in_pixels vs random).

Figure 3. Reconstructions for $k = 5$ (in_pixels vs random).

Figure 4. Reconstructions for $k = 7$ (in_pixels vs random).

1. Introduction

1.1. Student Information

Full name	Nguyễn Thế Hiển
Student ID	22127107
Class	22CLC08
Course	MTH00051 — Applied Mathematics and Statistics
Project	Project 01 — Color Compression

1.2. Motivation and Objectives

A color photograph is stored as a rectangular array of pixels. In the RGB model, every pixel carries three channel intensities, each typically encoded by one byte. The resulting discrete color cube $\{0,1,\dots,255\}^3$ contains up to $256^3 = 1.68 \times 10^7$ admissible colors. Natural images rarely use the entire cube, yet the number of observed colors is still large enough to inflate storage and transmission cost.

Color compression (also called color quantization) replaces the original palette by a much smaller set of prototypes. From the viewpoint of applied statistics, the task is unsupervised clustering in a three-dimensional feature space: similar colors are grouped, and each group is summarized by its centroid. The objectives of this practical report are therefore:

- 1 to formalize color quantization as an optimization problem on pixel vectors;
- 2 to implement K-Means from scratch with the two prescribed initializations;
- 3 to provide an interactive program that exports reconstructed images as png, jpg, or pdf;
- 4 to evaluate visual quality for several palette sizes and discuss the results rigorously.

2. Problem Formulation

Let an RGB image of height H and width W be represented by the collection of pixel vectors

$$X = \{x_i \in \mathbb{R}^3 \mid i = 1, \dots, N\}, \quad N = H \cdot W$$

(1)

Given an integer palette size k , one seeks centroids $C = \{c_1, \dots, c_k\} \subset \mathbb{R}^3$ and an assignment map $\ell : \{1, \dots, N\} \rightarrow \{1, \dots, k\}$ that minimize the within-cluster sum of squares (WCSS)

$$J(C, \ell) = \sum_{i=1}^N \|x_i - c_{\ell(i)}\|_2^2$$

(2)

The compressed image is then reconstructed by $\hat{x}_i = c_{\ell(i)}$. Consequently, at most k distinct colors appear in the output. Reconstruction fidelity in RGB geometry may be measured by the mean squared error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|_2^2 = \frac{J(C, \ell)}{N} \quad (3)$$

Equation (3) coincides with the normalized objective (2). Lower MSE indicates a closer Euclidean approximation; perceptual quality, however, also depends on the human visual system and is therefore examined visually in Section 5.

3. Theoretical Background

3.1. K-Means as an Optimization Procedure

Exact global minimization of (2) is NP-hard in general. The classical Lloyd iteration therefore alternates two conditionally optimal steps.

Assignment step. For fixed centroids, every pixel is labeled by the nearest prototype under the squared Euclidean metric:

$$\ell(i) = \arg \min_{j \in \{1, \dots, k\}} \|x_i - c_j\|_2^2 \quad (4)$$

Update step. For a fixed assignment, each nonempty cluster is summarized by its sample mean—the unique minimizer of the quadratic loss on that subset:

$$c_j \leftarrow \frac{1}{|S_j|} \sum_{i \in S_j} x_i, \quad S_j = \{i \mid \ell(i) = j\} \quad (5)$$

If a cluster becomes empty, the implementation retains the previous centroid to preserve numerical stability. Iteration stops when centroids change by less than a prescribed absolute tolerance (10^4 in this project) or when a maximum iteration budget is exhausted. Because neither step increases J , the sequence of objective values is monotonically non-increasing and converges to a local minimum.

3.2. Centroid Initialization Strategies

K-Means is sensitive to the initial configuration of centroids. Following the assignment statement, two schemes are implemented.

random. Each centroid coordinate is drawn independently and uniformly from $[0, 255]$. This explores the full RGB cube and may place prototypes outside the empirical color support of the image, occasionally producing empty clusters in early iterations.

in_pixels. Centroids are sampled uniformly without replacement from observed pixels. The initial palette therefore lies inside the data cloud, which typically yields a more favorable starting value of J and more stable early assignments.

Neither strategy guarantees a global optimum. Comparing both initializations for the same k provides empirical insight into the variability of local solutions.

4. Implementation Idea and Description

4.1. Software Environment

The solution is developed in a Jupyter Notebook and uses only the libraries permitted by the course specification. Pre-built clustering routines (for example, `sklearn.cluster.KMeans`) are excluded from the submission, although they may be used privately for cross-checking.

Library	Purpose in this project
NumPy	Matrix computation: distance evaluation, centroid means, reshape, and MSE.
Pillow (PIL)	Image reading/writing, RGB conversion, and export to png / jpg / pdf.
Matplotlib	Display of original and reconstructed images during testing and demos.

4.2. Implementation Idea (K-Means Recipe)

Following the classical Lloyd procedure, the practical pipeline used in this project can be summarized as the following recipe:

- 1 **Choose k .** Select the desired number of color clusters (palette size).
- 2 **Initialize centroids.** Create k initial prototypes either uniformly in $[0, 255]^3$ (*random*) or by sampling actual image pixels (*in_pixels*).
- 3 **Compute distances.** For every pixel, evaluate the squared Euclidean distance to each of the k centroids in RGB space.
- 4 **Assign clusters.** Label each pixel by the index of the nearest centroid.
- 5 **Update centroids.** Replace each centroid by the mean of all pixels assigned to that cluster (retain the old centroid if the cluster is empty).
- 6 **Iterate until convergence.** Repeat the distance–assignment–update cycle until centroids stabilize within tolerance or the iteration budget is exhausted.
- 7 **Reconstruct the image.** Replace every pixel by its centroid color and reshape the array back to $(H, W, 3)$, then save the result.

Operationally, an RGB image is first flattened into an $(N, 3)$ matrix so that clustering acts on color vectors only; spatial coordinates are ignored. After clustering, the label map restores spatial structure.

4.3. Detailed Function Descriptions

This subsection documents every major function in the notebook in the same spirit as a technical specification: purpose, parameters, returns, and algorithmic steps.

4.3.1. `read_img(img_path)`

Purpose. Read an image from disk and convert it into a NumPy array of shape $(H, W, 3)$.

Parameters.

- `img_path` (str): path to the input image file.

Returns. A floating-point RGB array `image_2d`.

Algorithm.

- 1 Open the file with `PIL . Image . open`.
- 2 Convert explicitly to RGB to avoid RGBA/palette inconsistencies.
- 3 Cast to a NumPy array with dtype `float64` for stable clustering arithmetic.

4.3.2. `show_img(img_2d)`

Purpose. Display a 2D image for visual inspection.

Parameters. `img_2d`: image array to display.

Algorithm. Clip intensities to `[0, 255]`, cast to 8-bit integers, call `matplotlib.pyplot.imshow`, and turn axes off.

4.3.3. `save_img(img_2d, img_path)`

Purpose. Persist a reconstructed image to disk.

Parameters.

- `img_2d`: image array to save;
- `img_path` (str): destination path, including extension.

Algorithm. Clip to `[0, 255]`, convert to `uint8`, wrap with `PIL.Image.fromarray`, and call `save`. The extension determines the container (`png` / `jpg` / `pdf`).

4.3.4. `convert_img_to_1d(img_2d)`

Purpose. Flatten a spatial image into a design matrix of pixel colors.

Parameters. `img_2d` with shape $(H, W, 3)$.

Returns. Array of shape $(N, 3)$ where $N = H \cdot W$.

Algorithm. Apply `reshape((-1, n_channels))` so that each row is one RGB vector ready for clustering.

4.3.5. `kmeans(img_1d, k_clusters, max_iter, init_centroids)`

Purpose. Perform K-Means on pixel colors to obtain a reduced palette and a label for every pixel.

Parameters.

- `img_1d` (ndarray, shape (N, c)): flattened image;
- `k_clusters` (int): number of clusters k ;
- `max_iter` (int): maximum Lloyd iterations;
- `init_centroids` (str): 'random' or 'in_pixels'.

Returns.

- `centroids` (ndarray, shape (k, c)): cluster color prototypes;
- `labels` (ndarray, shape $(N,)$): cluster index of each pixel.

Algorithm.

- 1 **Initialize.** If random, sample each centroid coordinate uniformly from `[0, 255]`; if `in_pixels`, sample k distinct pixels without replacement.
- 2 **Assign.** Compute squared Euclidean distances between all pixels and all centroids; set each label to the nearest centroid index (Eq. 4).
- 3 **Update.** Replace each nonempty cluster's centroid by the mean of its members (Eq. 5); keep the previous centroid if empty.
- 4 **Stop.** Halt when `np.allclose` reports centroid changes below 10^4 , or when `max_iter` is reached.

4.3.6. `generate_2d_img(img_2d_shape, centroids, labels)`

Purpose. Reconstruct a spatial image from centroids and pixel labels.

Parameters.

- `img_2d_shape` (tuple): original $(H, W, 3)$;
- `centroids`: palette colors;
- `labels`: per-pixel cluster indices.

Returns. New 2D image with at most k colors, dtype `uint8`.

Algorithm. Index `centroids[labels]` to replace every pixel by its prototype, reshape to the original geometry, clip to $[0, 255]$, and cast to 8-bit.

4.3.7. `reconstruction_error(img_1d, centroids, labels)`

Purpose. Compute the mean squared error between original pixels and their reconstructed colors (Eq. 3), providing a quantitative complement to visual assessment.

Algorithm. Form `centroids[labels]` and return the mean of squared coordinate-wise differences.

4.4. Testing Procedure

The helper `test_functions(img_path)` performs an end-to-end smoke test of the pipeline. When executed on a valid image it:

- reads the image and asserts that the array is a valid 3-channel RGB tensor;
- displays the original image;
- flattens the image and checks that the resulting shape is $(H \cdot W, 3)$;
- runs K-Means with both *random* and *in_pixels* initializations (default $k = 5$);
- asserts that centroid and label shapes match the specification;
- reconstructs images, displays them, reports MSE, and saves png/pdf previews;
- prints `All tests passed.` upon success.

This procedure mirrors the validation workflow used during development and documents that every stage—I/O, reshaping, clustering, reconstruction, and export—operates consistently.

4.5. Main Program Workflow

The interactive main procedure satisfies the course interface requirements. At each execution the user supplies:

- the path of the input image;
- the number of clusters k and the maximum number of iterations;
- the initialization mode (*random*, *in_pixels*, or both);
- the output format chosen from {png, jpg, pdf};
- a base filename for the reconstructed image(s).

The pipeline then flattens the image, runs K-Means for each requested initialization, displays the reconstruction, reports the number of unique colors and the MSE, and exports the file in the selected format. Running both initializations in one session facilitates the side-by-side comparison presented in Section 5.

5. Experimental Results

5.1. Experimental Setup

Experiments are conducted on a natural beach photograph containing smooth illumination gradients (sky and water) together with a high-contrast silhouette. Such content is informative for quantization: coarse palettes posterize the sky, while larger palettes restore intermediate tones. Both initialization strategies are evaluated for $k \in \{3, 5, 7\}$ with a sufficient iteration budget to reach practical convergence.

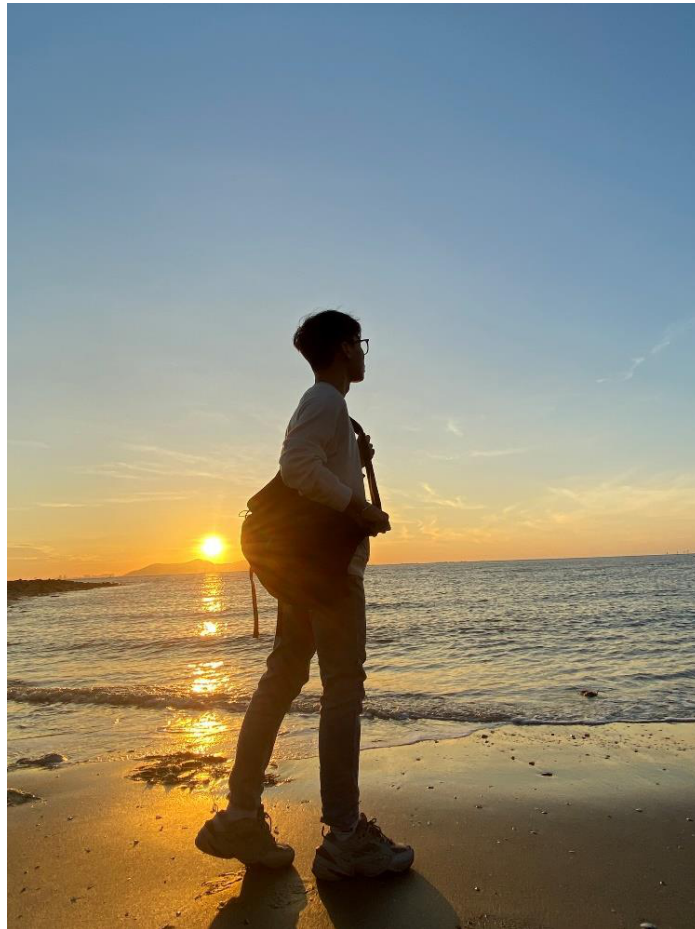


Figure 1. Original test image prior to color compression.

5.2. Visual Comparison for $k = 3, 5, 7$

For $k = 3$ (Figure 2), both initializations reduce the scene to a silhouette, a mid-tone band, and a sky tone. Fine water reflections are largely suppressed, yet the semantic layout remains recognizable. The close agreement of the two methods suggests that the dominant chromatic modes are stable under different starts.

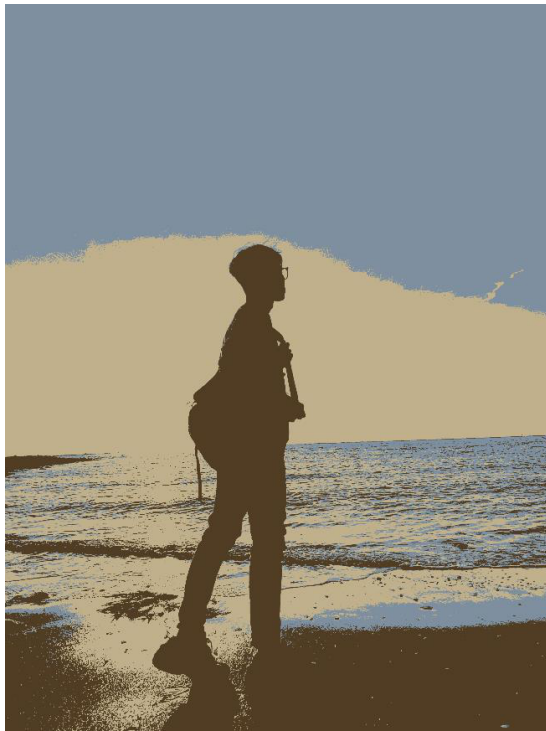


Figure 2a. *in_pixels* initialization, $k = 3$.

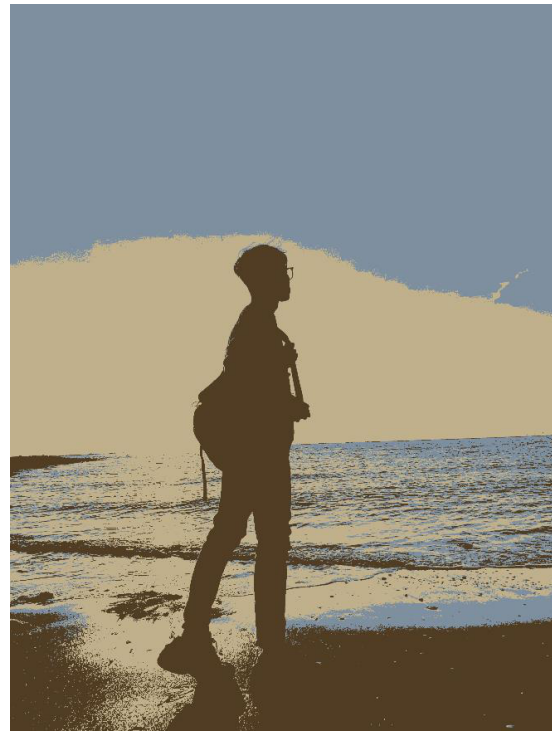


Figure 2b. *random* initialization, $k = 3$.

For $k = 5$ (Figure 3), additional centroids capture transitional hues near the horizon and richer reflections. Differences between initialization schemes become more noticeable: random sampling may allocate a centroid to a less frequent region of RGB space, slightly altering the tonal balance relative to *in_pixels*.

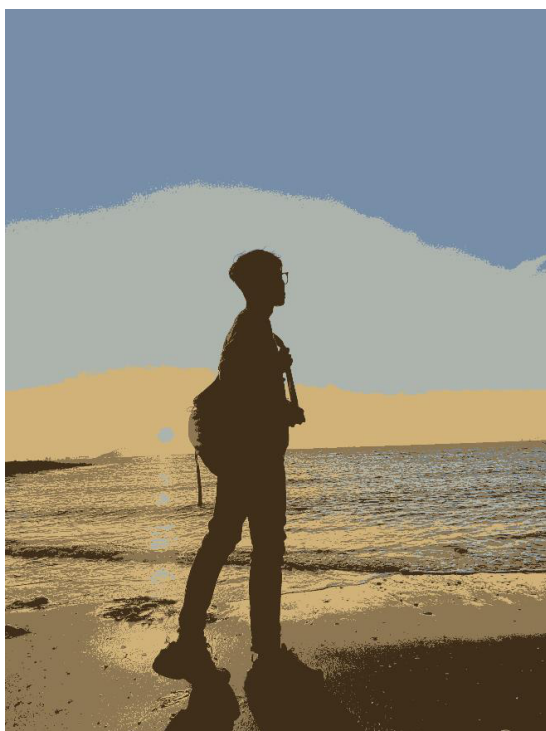


Figure 3a. *in_pixels* initialization, $k = 5$.



Figure 3b. *random* initialization, $k = 5$.

When $k = 7$ (Figure 4), reconstructions approach the original more closely. Gradients in the sky and specular highlights on wet sand reappear as distinct palette entries. Residual discrepancies between the two initializations illustrate that K-Means may converge to distinct local minima even for moderate palette sizes.

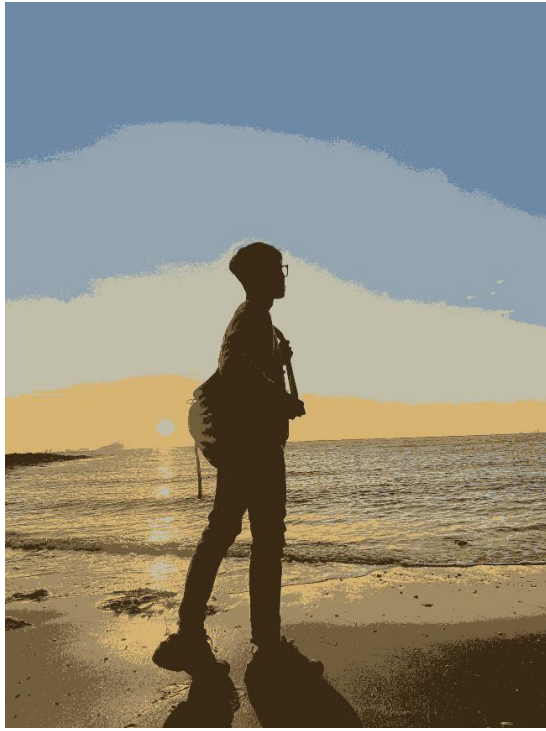


Figure 4a. *in_pixels* initialization, $k = 7$.

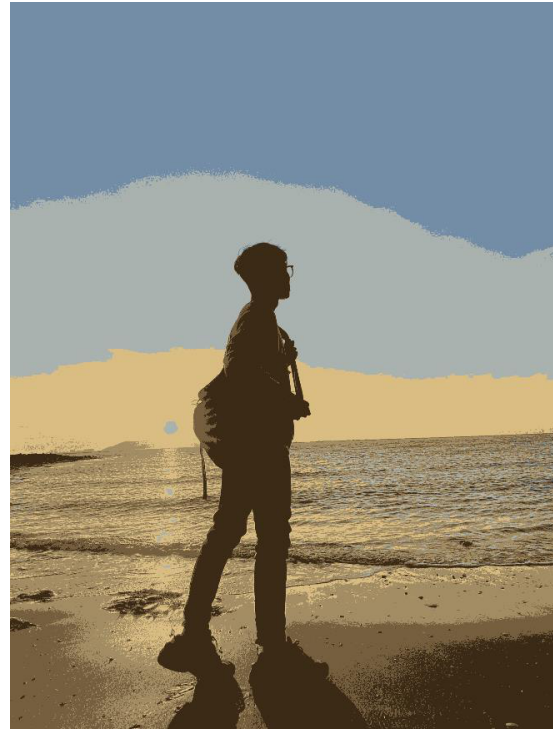


Figure 4b. *random* initialization, $k = 7$.

5.3. Qualitative Summary

k	Visual fidelity	Effect of initialization
3	Strong posterization; content preserved only at a coarse semantic level.	Minor differences; dominant colors are recovered consistently.
5	Improved mid-tones and horizon detail; water structure becomes clearer.	Noticeable palette shifts between random and <i>in_pixels</i> .
7	Closer to the original; smoother apparent gradients and richer reflections.	Local minima remain visible but are less disruptive perceptually.

6. Discussion

6.1. Results Commentary

Effect of palette size. The experiments confirm the classical complexity–fidelity trade-off of prototype methods. With $k = 3$ the image is strongly posterized: only the dominant silhouette, mid-tone, and sky modes survive. At $k = 5$, transitional horizon colors and richer water structure reappear. At $k = 7$ the reconstruction approaches the original more closely, with smoother apparent gradients. In mathematical terms, the feasible set of reconstructions expands with k , so the minimal attainable value of J is monotonically non-increasing in the palette size.

Random initialization. Centroids are drawn from the full RGB cube $[0, 255]^3$. This broad exploration can help discover separated modes in chromatic images, but early iterations may produce empty clusters when prototypes land far from the empirical color support. Visually, random starts sometimes shift the tonal balance relative to *in_pixels*, especially for moderate k .

In-pixels initialization. Centroids are sampled from observed pixels, hence they begin inside the data cloud. This usually yields a more favorable initial value of the WCSS and more stable early assignments. For the beach photograph studied here, `in_pixels` reconstructions often look slightly more natural in the horizon band, although the difference is not uniformly large.

6.2. Comparison of Initialization Strategies

A careful reading of the figures does **not** support blanket claims that one scheme is always faster or always superior in quality. Instead:

- `random` may be advantageous when the image occupies many well-separated regions of RGB space, because unconstrained sampling can place seeds near several modes;
- `in_pixels` is generally safer for images with concentrated or nearly uniform palettes, because every seed is a feasible color from the outset;
- for small k (e.g., 3), both methods often recover essentially the same dominant modes; divergence becomes more visible as k grows.

Thus the recommended experimental practice—implemented in `main`—is to run both initializations and compare reconstructions (and, when available, MSE) before choosing an export.

6.3. Limitations

Several limitations should be acknowledged. First, Euclidean distance in RGB is only an approximate proxy for perceptual dissimilarity; distances in CIELAB would align more closely with human judgment. Second, forming a full distance tensor has memory cost $O(Nk)$ and may require chunked evaluation for very large images. Third, a single photograph does not exhaust the diversity of natural scenes; a broader benchmark with repeated random seeds would strengthen statistical conclusions about initialization.

7. Conclusion

This practical exercise shows that K-Means clustering provides a clear and effective mechanism for color compression. By casting pixels as points in $\mathbb{R}^3$ and minimizing a quadratic distortion criterion, one obtains a reduced palette that retains the essential visual narrative of an image. The implemented system fulfills the project requirements: a from-scratch K-Means routine with both prescribed initializations, an interactive main program supporting png/jpg/pdf export, and a systematic qualitative evaluation for multiple values of k .

Natural extensions include K-Means++ initialization, multi-run selection of the best local minimum, and quantitative reporting of MSE/PSNR across a curated image set. Such enhancements would further connect the algorithmic prototype developed here with contemporary practice in statistical image analysis.

References

- [1] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- [2] Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

- [3] Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027–1035.
- [4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Chapter 9: Mixture models and EM; background on prototype-based clustering).
- [5] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://numpy.org/doc/>
- [6] Clark, A., and contributors. (n.d.). *Pillow (PIL Fork) Documentation*. <https://pillow.readthedocs.io/en/stable/>
- [7] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://matplotlib.org/stable/>